

1. Идея проекта и решаемая проблема

В современном мире у каждого человека есть множество аккаунтов: электронная почта, социальные сети, интернет-банкинг, рабочие системы, учебные порталы, форумы, стриминговые сервисы и десятки других. Для каждого требуется пароль. Использовать один и тот же пароль везде очень опасно: если злоумышленник узнает пароль от одного сервиса (например, из-за утечки базы данных), он получит доступ ко всем аккаунтам. Запоминать 20-30 разных сложных паролей человеческая память не позволяет.

Менеджеры паролей (LastPass, Bitwarden, 1Password) хранят все пароли в одном зашифрованном месте, но это создает единую точку отказа. Если злоумышленник получит доступ к мастер-паролю от менеджера, все пароли окажутся скомпрометированы. Кроме того, менеджеры требуют установки дополнительного программного обеспечения, синхронизации между устройствами и доверия к облачному хранилищу.

passgen предлагает принципиально другой подход — детерминированную генерацию. Программа не хранит пароли вообще. Вместо этого она каждый раз вычисляет пароль заново с помощью криптографического алгоритма HMAC-SHA256. Достаточно запомнить одну секретную фразу (сид) и знать, для какого сервиса нужен пароль. Программа всегда выдаст один и тот же пароль для одинаковых входных данных. Это означает, что пароль можно восстановить на любом устройстве, в любой момент времени, без необходимости носить с собой базу данных паролей или синхронизировать её через облако.

Преимущества такого подхода очевидны: нет единой точки отказа, нет необходимости в облачном хранилище, нет риска кражи базы паролей. Даже если устройство будет полностью уничтожено, все пароли останутся доступны — нужно лишь вспомнить сид. При этом для каждого сервиса создаётся уникальный, криптостойкий пароль, который невозможно подобрать или предсказать.

2. Структура и архитектура программы

Программа состоит из одного HTML-файла (index.html), который содержит всё необходимое: разметку (HTML), стили (CSS) и логику (JavaScript). Для работы программы не требуется устанавливать дополнительные библиотеки, пакеты или программы. Достаточно любого современного браузера с поддержкой Web Crypto API (Google Chrome, Mozilla Firefox, Microsoft Edge, Safari, Opera, Yandex Browser и другие). Программа полностью работает на стороне клиента, никакие данные не передаются по сети и не сохраняются на сервере.

2.1. HTML — разметка страницы

HTML-код определяет структуру интерфейса. Основные элементы: поле ввода сида (секретной фразы), поле ввода названия сервиса, панель с популярными сервисами, выпадающий список пресетов, регулятор длины пароля, переключатели типов символов (заглавные, строчные, цифры, спецсимволы), расширенные настройки (первая заглавная, последняя цифра), кнопка генерации, поле вывода логина и пароля, индикатор надежности, кнопка копирования, ссылки экспорта и импорта, модальное окно импорта. Каждый элемент имеет уникальный идентификатор (id) для доступа из JavaScript.

2.2. CSS — стили и оформление

Оформление программы выполнено в современном минималистичном стиле с использованием CSS-переменных (custom properties). Цветовая схема: светло-фиолетовый фон (#f5f3ff), белые карточки с тенью, темный текст, фиолетовые акценты. Все кнопки имеют градиентную заливку, скругленные углы и тени. Реализованы анимации: fadeUp для плавного появления элементов, glow для пульсации ошибки, slideDown/slideUp для раскрытия панелей, checkPop для анимации переключателей. Переключатели символов выполнены в виде кружочков (кастомные чекбоксы). Стрелки регулятора длины вынесены отдельно от поля ввода.

2.3. JavaScript — логика программы

Вся логика программы написана на JavaScript с использованием Web Crypto API — встроенной в браузер криптографической библиотеки. Это означает, что все криптографические операции (хеширование, HMAC)

выполняются нативным кодом браузера, а не интерпретируются JavaScript. Программа содержит 26 функций, каждая из которых отвечает за определённую задачу: генерацию пароля, управление настройками, экспорт/импорт, отрисовку интерфейса и т.д.

3. Криптографический алгоритм

3.1. Что такое HMAC-SHA256 и чем отличается от обычного хеша

SHA-256 (Secure Hash Algorithm 256 бит) – это криптографическая хеш-функция, которая принимает входные данные произвольной длины и возвращает фиксированный 256-битный (32-байтный) дайджест. Хеш-функция обладает следующими

свойствами:

необратимость

(по

хешу

невозможно

восстановить

исходные

данные),

детерминированность (одинаковые входные данные всегда дают одинаковый хеш), лавинный эффект (изменение одного бита на входе меняет в среднем половину битов на выходе), устойчивость к коллизиям (крайне сложно найти два разных сообщения с одинаковым хешем).

HMAC (Hash-based Message Authentication Code) – это конструкция, которая использует хеш-функцию вместе с секретным ключом. Если провести аналогию: SHA-256 – это просто чёрный ящик, который превращает любые данные в 32 байта. Любой может вычислить SHA-256 от любого сообщения. HMAC – это сейф с ключом: чтобы вычислить HMAC, нужно знать секретный ключ. Даже если алгоритм HMAC публичный, без знания ключа невозможно вычислить правильный HMAC от сообщения.

В программе passgen HMAC-SHA256 используется следующим образом: секретная фраза пользователя (сид) становится ключом HMAC, а сообщением – строка, составленная из названия сервиса и длины пароля. Такой подход гарантирует, что: (1) без знания сида невозможно вычислить пароль; (2) для разных сервисов получаются совершенно разные пароли; (3) для одного и того же сервиса пароль всегда одинаков (детерминированность); (4) даже небольшое изменение сида полностью меняет все пароли.

3.2. Почему именно HMAC, а не просто SHA-256 от seed + service

Если бы программа использовала просто SHA-256(seed + service), это было бы уязвимо к атаке добавления (length extension attack). SHA-256 имеет уязвимость: зная SHA-256(M), можно вычислить SHA-256(M + padding + extra) без знания M. HMAC не подвержен этой атаке, так как использует ключ дважды: сначала применяется XOR с внутренним паддингом, хешируется с сообщением, затем результат хешируется с ключом после XOR с внешним паддингом. Конструкция HMAC доказано безопасна при условии, что хеш-функция обладает криптографической стойкостью.

3.3. Rejection Sampling – обеспечение равномерного распределения

При превращении байтов HMAC в символы пароля возникает математическая проблема. Каждый байт, полученный от HMAC, может принимать значение от 0 до 255 (256 различных значений). Длина алфавита (набора символов для пароля) может быть, например, 50 или 62. Число 256 не делится нацело ни на 50, ни на 62.

Если бы программа просто брала остаток от деления (байт % длина_алфавита), некоторые символы встречались бы чаще других. Рассмотрим пример: алфавит из 50 символов. Байты от 0 до 249 (250 значений) дают остатки от 0 до 49 – каждый символ встречается по 5 раз. Но байты от 250 до 255 (6 значений) дают остатки от 0 до 5 – символы 0-5 получают дополнительное вхождение. Это создаёт смещение (bias), которое недопустимо в криптографии:

злоумышленник может использовать эту неравномерность для ускорения подбора пароля.

Решение, применяемое в программе: $\text{max} = 256 - (256 \% \text{длина_алфавита})$. Все байты, значение которых больше или равно max, отбрасываются (reject). Используются только байты со значением меньше max. Поскольку max кратен длине алфавита, распределение символов становится строго равномерным. Пример: алфавит из 50 символов, $\text{max} =$

250. Байты 250-255 отбрасываются. Каждый символ встречается ровно 5 раз. Вероятность каждого символа – 1/50.

Этот метод называется Rejection Sampling – стандартная техника в криптографии. Недостаток метода: теоретически может потребоваться больше одной итерации HMAC, если все 32 байта оказались отбракованы. На практике вероятность отбраковки одного байта составляет $6/256 = 2.3\%$, а вероятность отбраковки всех 32 байтов ничтожно мала.

4. Пользовательский интерфейс

Интерфейс программы состоит из следующих элементов, расположенных сверху вниз:

4.1. Заголовок и кнопки экспорта/импорта

В верхней части страницы расположен заголовок 'passgen' и две функциональные ссылки: 'Экспорт' (выгрузка настроек в JSON-файл) и 'Импорт' (загрузка настроек из JSON-файла). При нажатии на 'Импорт' открывается системное окно выбора файла.

4.2. Карточка "Доступ" (сид)

Поле ввода сида – секретной фразы, которую пользователь держит в голове. Под полем отображается пояснение: 'Секретная фраза, которую вы держите в голове.' Если сид не введён при попытке генерации пароля, поле подсвечивается нежно-розовой обводкой с анимацией пульсации, а под полем появляется сообщение 'Введите сид'. Ошибка исчезает автоматически при начале ввода.

4.3. Карточка "Сервис"

Поле ввода названия сервиса, для которого генерируется пароль (например, google, github, vk). Справа от заголовка расположена ссылка 'список' с иконкой стрелки, которая открывает панель с предустановленными популярными сервисами. Каждый сервис отображается в виде чипа (кнопки) с логотипом (буквенная аббревиатура на цветном фоне). При клике на чип название сервиса подставляется в поле ввода, а панель закрывается. Если пользователь вводит свой сервис, которого нет в списке, при первой генерации пароля он автоматически добавляется в список сохранённых сервисов (отображается без логотипа, с вопросительным знаком). Сохранённые сервисы можно удалить по правому клику мыши (ПКМ).

4.4. Карточка "Параметры"

Содержит выпадающий список пресетов (Свой, Минимальный, Стандартный, Максимальный), регулятор длины пароля (числовое поле от 1 до 99 с кнопками +/-), переключатели типов символов (Заглавные, Строчные, Цифры, Спецсимволы) в виде кружочков, и расширенные настройки (Первая заглавная, Последняя цифра), которые скрыты под сворачиваемой панелью.

4.5. Кнопка "Создать"

Главная кнопка генерации пароля. При нажатии запускается функция gen(), которая проверяет корректность ввода, запускает HMAC-генерацию, применяет инверсии, выводит результат и автоматически копирует его в буфер обмена.

4.6. Карточка "Логин и пароль"

Содержит поле вывода логина (с кнопкой его генерации) и поле вывода пароля (только для чтения). Под паролем расположен индикатор надёжности: три полосы, окрашиваемые в зависимости от качества пароля, и текстовое описание с эмодзи.

4.7. Кнопка "Копировать"

Дублирующая кнопка копирования пароля в буфер обмена. Под ней отображается сообщение 'Скопировано!' на 1.5-2 секунды после успешного копирования.

4.8. Модальное окно импорта

При импорте JSON-файла, если в нём содержатся данные для нескольких сервисов, появляется модальное окно, в котором можно выбрать конкретные сервисы для импорта. По умолчанию все сервисы отмечены.

5. Описание каждой функции программы

В этом разделе представлено подробное описание всех 26 функций программы. Каждая функция рассмотрена с точки зрения её назначения, принципа работы, входных и выходных данных, а также особенностей реализации.

5.1. Функция `gen()` — генерация пароля

Назначение: главная функция всей программы, которая запускает процесс создания пароля. Вызывается при нажатии кнопки 'Создать' или клавиши Enter в полях сیدا или сервиса.

Порядок работы функции:

1)

Получение

данных

из

полей

ввода.

Функция

считывает

значение

из

поля

сیدا

(`document.getElementById('seed').value`), обрезает пробелы по краям (`trim()`). Если сід пустой, функция добавляет полю класс 'error' (розовая подсветка с пульсацией), выводит сообщение 'Введите сід' под полем и завершает работу.

2) Получение названия сервиса и длины. Сервис считывается из соответствующего поля; если поле пустое, используется значение '(empty)'. Длина считывается из поля `length`; если значение некорректно, используется значение по умолчанию 10.

3) Сбор набора символов. Функция проверяет, какие типы символов включены (`capitals`, `lowercase`, `digits`, `symbols`). Если ни один тип не выбран, выводится сообщение 'Выберите хотя бы один тип символов' и работа прекращается. Из выбранных типов формируется алфавит — строка, содержащая все разрешённые символы.

4) Генерация пароля. Вызывается асинхронная функция `hmacGen()`, которой передаются сід, сервис, длина и алфавит. Функция `hmacGen()` возвращает сгенерированный пароль.

5) Применение инверсий. Если включена опция 'Первая заглавная' (`inv.cap`), первый символ пароля преобразуется в заглавный (`toUpperCase()`). Если включена опция 'Последняя цифра' (`inv.dig`), последний символ пароля заменяется на цифру. Цифра вычисляется детерминированно: создаётся новый HMAC-ключ из сیدا+'_lastdig', подписывается сообщение сервис+длина, первый байт результата берётся по модулю 10. Такой подход гарантирует, что последняя цифра всегда одинакова для одних и тех же входных данных.

6) Вывод результата. Полученный пароль отображается в поле `pwd` (только для чтения). Запускается функция `updateStrength()` для обновления индикатора надёжности.

7) Автоматическое копирование. Пароль автоматически копируется в буфер обмена `navigator.clipboard.writeText()`. Под кнопками на 2 секунды появляется сообщение 'Скопировано!'.

с

помощью

8) Сохранение сервиса. Если введённый сервис отсутствует в списке сохранённых и не равен '(empty)', он добавляется в массив `svcs`, список сортируется, и чипы перерисовываются.

9) Сохранение настроек. Вызывается `saveCfg()`, которая записывает текущие настройки (состояние переключателей, инверсии, длину) для текущего сервиса.

Важно: функция `gen()` является асинхронной (`async`), так как `hmacGen()` использует `await` для ожидания результата криптографических операций.

5.2. Функция hmacGen() – криптографическая генерация пароля

Назначение: ядро программы, реализующее детерминированную генерацию пароля на основе HMAC-SHA256 с использованием Rejection Sampling для равномерного распределения символов.

Входные параметры:

- seed (string) – секретная фраза пользователя, становится ключом HMAC.
- svc (string) – название сервиса, участвует в формировании сообщения HMAC.
- len (number) – требуемая длина пароля.
- alpha (string) – алфавит (строка всех разрешённых символов).

Выходное значение: строка пароля указанной длины.

Алгоритм работы:

- 1) Подготовка ключа. Секретная фраза (сид) кодируется в байты с помощью `TextEncoder()`. На основе этих байтов создаётся криптографический ключ HMAC-SHA256 с помощью `crypto.subtle.importKey('raw', keyBytes, {name:'HMAC', hash:'SHA-256'}, false, ['sign'])`. Параметр `false` означает, что ключ нельзя экспортировать (безопасность).
- 2) Подготовка сообщения. Строка, состоящая из названия сервиса и длины (`svc + len`), кодируется в байты – это базовая часть сообщения.
- 3) Вычисление `max` для Rejection Sampling. `max = 256 - (256 % alpha.length)`. Все байты HMAC, значение которых $\geq \text{max}$, отбрасываются.
- 4) Генерация символов в цикле. Создаётся счётчик `ct = 0`. В цикле `while (pwd.length < len)` к сообщению добавляется байт счётчика (`ct++`), вычисляется HMAC от этого сообщения. Полученные 32 байта хеша перебираются: если байт $< \text{max}$, он преобразуется в символ по формуле `alpha[b % alpha.length]`. Символ добавляется к паролю. Если длина пароля достигла требуемой, цикл завершается.
- 5) Если после перебора всех 32 байтов пароль всё ещё короче нужной длины, счётчик увеличивается и цикл повторяется с новым значением счётчика. Таким образом, каждое новое обращение к HMAC использует уникальное сообщение, что гарантирует получение новых 32 байтов.

Криптографическая стойкость: функция `hmacGen()` обеспечивает следующие свойства: (1) детерминированность – одинаковые входные данные всегда дают одинаковый пароль; (2) равномерность – благодаря Rejection Sampling все символы алфавита встречаются с одинаковой вероятностью; (3) необратимость – по паролю невозможно восстановить сид; (4) лавинный эффект – изменение любого входного параметра полностью меняет пароль; (5) стойкость к коллизиям – вероятность совпадения паролей для разных входных данных пренебрежимо мала.

5.3. Функция genLogin() – генерация логина

Назначение: создание простого, легко запоминающегося логина (имени пользователя) на основе сида и названия сервиса.

Порядок работы:

- 1) Очистка поля логина. Функция начинает с очистки поля вывода логина.
- 2) Проверка ввода. Если сид не введён, функция подсвечивает поле сида ошибкой (как и `gen()`) и выводит сообщение 'Введите сид'. Если сервис не введён, функция просто завершается без ошибки.
- 3) Вычисление HMAC. Сид используется как ключ HMAC, сообщением является строка `'login/' + svc`. Вызывается `crypto.subtle.sign('HMAC', k, enc.encode('login/' + svc))`, где `k` – HMAC-ключ, созданный из сида.
- 4) Формирование логина. Из 32 байтов результата HMAC берутся первые 6 байтов. Каждый байт берётся по модулю длины алфавита (`alpha.length`). Алфавит для логинов специально подобран: `'abcdefghijklmnopqrstuvwxyz23456789'`. Из него исключены символы, которые легко перепутать: `'i'` и `'l'` (похожи на `'1'`), `'o'` (похож на `'0'`), `'0'` и `'1'` (похожи на буквы). Таким образом, логин состоит из 6 букв и цифр, которые легко продиктовать и запомнить.
- 5) Вывод результата. Полученный 6-символьный логин отображается в поле `login`.

Особенность: функция `genLogin()` использует HMAC-SHA256, где сид является ключом, а сообщением – строка `'login/' + svc`. Логин детерминирован: одинаковый сид + сервис всегда дают одинаковый логин.

5.4. Функция updateStrength() – индикатор надёжности пароля

Назначение: визуальная оценка качества сгенерированного пароля на основе двух критериев: длины и количества используемых типов символов.

Порядок работы:

- 1) Очистка индикатора. Функция очищает контейнер `strBars` (удаляет все полосы) и получает ссылку на элемент `strInfo`

для текстового описания.

2) Определение уровня надежности. Используются следующие пороги:

- Сильный пароль (strong): длина ≥ 14 И количество типов символов ≥ 4 . Отображается три сиреневые полосы, текст 'Отличный пароль', эмодзи 'uwi'.
- Средний пароль (medium): длина ≥ 10 И количество типов символов ≥ 3 . Отображается две малиновые полосы, текст 'Хороший, можно длиннее', эмодзи ':3'.
- Слабый пароль (weak): все остальные случаи. Отображается одна розовая полоска, текст 'Добавьте символов или увеличьте длину', эмодзи '>.<'.

3) Отрисовка полосок. Создаются три div-элемента с классом 'str-bar'. Каждой полоске в зависимости от уровня добавляются классы 'on' и цветовой класс (weak/medium/strong). Полоски, не соответствующие уровню, остаются серыми.

4) Вывод текста. Текстовый элемент получает соответствующий класс окраски и отображает эмодзи и сообщение. Назначение цветов: розовый для слабых паролей предупреждает пользователя; малиновый (crimson) для средних паролей мотивирует улучшить; сиреневый (lilac) для сильных паролей показывает, что пароль отличный.

5.5. Функция сру() – копирование в буфер обмена

Назначение: копирование сгенерированного пароля в буфер обмена для последующей вставки в поле пароля на сайте или в приложении.

Порядок работы: функция считывает значение из поля pwd. Если поле пустое, функция завершается. Если поле не пустое, предпринимается попытка скопировать текст с помощью современного API navigator.clipboard.writeText(). Это асинхронный метод, который возвращает Promise. Если этот метод недоступен или вызывает ошибку (например, в некоторых старых браузерах или при отсутствии HTTPS), используется запасной метод: текст в поле выделяется (select()) и копируется через execCommand('copy').

После копирования под кнопками на 1.5 секунды отображается сообщение 'Скопировано!'. Сообщение исчезает автоматически. Если копирование не удалось, сообщение не показывается.

5.6. Функция saveCfg() – сохранение настроек сервиса

Назначение: сохранение текущих настроек (состояние переключателей, инверсии и длины) для текущего сервиса в оперативной памяти.

Порядок работы: функция создаёт или обновляет запись в объекте perSvc для текущего сервиса (curSvc).

Сохраняются три параметра: state (объект с флагами capitals, lowercase, digits, symbols), inv (объект с флагами cap, dig), len (числовое значение длины). Все значения копируются через spread-оператор (...window.state, ...window.inv), чтобы создать копии, а не ссылки на оригинальные объекты. После сохранения вызывается функция saveState() (в текущей версии пустая, ранее сохраняла данные в localStorage).

Функция saveCfg() вызывается: (1) после успешной генерации пароля в конце gen(); (2) после применения пресета в applyPreset(). Она не вызывается при каждом изменении переключателей – только в ключевые моменты.

5.7. Функция getCfg() – получение настроек сервиса

Назначение: получение настроек для указанного сервиса. Если настройки ещё не были сохранены, функция создаёт их со стандартными значениями.

Входной параметр: svc (string) – название сервиса.

Выходное значение: объект настроек сервиса (perSvc[svc]). Если сервис отсутствует в perSvc, создаётся новый объект с настройками по умолчанию: capitals=1, lowercase=1, digits=0, symbols=1, inv.cap=0, inv.dig=0, len=10. Это гарантирует, что для нового сервиса всегда будут разумные настройки по умолчанию.

5.8. Функция applyCfg() – применение настроек сервиса

Назначение: применение сохранённых настроек к интерфейсу программы.

Входной параметр: cfg (object) – объект настроек сервиса.

Порядок работы: функция устанавливает window.state = cfg.state, window.inv = cfg.inv, значение поля длины – cfg.len. Затем вызываются renderChk() для обновления отображения переключателей символов и renderInv() для обновления отображения инверсий. Таким образом, при переключении между сервисами интерфейс автоматически настраивается под каждый сервис.

5.9. Функция init() – инициализация программы

Назначение: запускается при загрузке страницы (последняя строка скрипта: `init()`). Выполняет начальную настройку программы: загрузку состояния, установку обработчиков событий и применение настроек для пустого сервиса.

Порядок работы:

- 1) Вызов `loadState()` для загрузки данных (в текущей версии – очистка всех данных).
- 2) Сброс ошибок: у поля сйда удаляется класс `'error'`, очищается текст ошибки.
- 3) Установка обработчиков событий:
 - `input` на поле сйда – при вводе текста убирается подсветка ошибки.
 - `keydown` на поле сйда – при нажатии `Enter` вызывается `gen()`.
 - `keydown` на поле сервиса – при нажатии `Enter` вызывается `gen()`.
 - `blur` на поле сервиса – при потере фокуса проверяется, изменился ли сервис; если изменился, обновляется `curSvc`, и если для сервиса есть настройки – они применяются, иначе создаются настройки на основе текущего состояния интерфейса.
- 4) Применение настроек: если для текущего сервиса (`curSvc`) есть настройки, они применяются. Иначе `curSvc` устанавливается в `'(empty)'`, поле сервиса очищается, и устанавливаются стандартные настройки по умолчанию.
- 5) Отрисовка чипов сервисов вызовом `renderSvcChips()`.

- 10 -

5.10. Функция `loadState()` – загрузка состояния (очистка данных)

Назначение: очистка всех ранее сохранённых данных при запуске программы. Реализует сессионный режим работы: при каждом открытии страницы программа начинает с чистого листа.

Порядок работы:

- 1) Попытка удалить ключ `'passgen_state'` из `localStorage`. Если ключ не существует, ошибка игнорируется благодаря `try/catch`.
- 2) Сброс объекта `perSvc` в пустой объект `{}`. Сброс массива `svcs` в пустой массив `[]`.
- 3) Установка `curSvc = '(empty)'`.
- 4) Установка `window.state` и `window.inv` в значения по умолчанию.
- 5) Установка длины пароля = 10, очистка поля сервиса.

Важно: функция намеренно очищает все данные при каждом запуске (сессионный режим). Это сделано для безопасности: старые настройки не сохраняются между сессиями, чтобы злоумышленник, получив доступ к компьютеру, не узнал, для каких сервисов пользователь создаёт пароли.

5.11. Функция `onSvcChange()` – обработчик смены сервиса

Назначение: вызывается при изменении значения в поле ввода сервиса. Обновляет текущий сервис и применяет его настройки, если они существуют.

Порядок работы: функция считывает значение из поля сервиса, обрезает пробелы. Если поле пустое, используется значение `'(empty)'`. `curSvc` устанавливается в полученное значение. Если для этого сервиса есть сохранённые настройки в `perSvc`, они применяются через `applyCfg()`.

5.12. Функция `saveState()` – заглушка безопасности

Назначение: в текущей версии пустая функция. Ранее она сохраняла настройки в `localStorage` для сохранения между сессиями. После перехода на сессионный режим (очистка всех данных при запуске) функция оставлена пустой для совместимости. Она вызывается из `saveCfg()`, но не выполняет никаких действий.

5.13. Функция `toggleInv()` – переключение инверсий

Назначение: переключение расширенных опций 'Первая заглавная' (`cap`) и 'Последняя цифра' (`dig`). Вызывается при клике на соответствующий пункт в панели расширенных настроек.

Входной параметр: `id (string)` – идентификатор опции (`'cap'` или `'dig'`).

Порядок работы: значение `window.inv[id]` инвертируется (`0->1`, `1->0`) с помощью выражения `!window.inv[id]`. Затем обновляется класс элемента кружочка: если значение стало 1 (истина), добавляется класс `'on'` (кружок закрашивается), иначе класс `'on'` удаляется (кружок пустой).

- 11 -

5.14. Функция `applyPreset()` – применение пресетов

Назначение: быстрая настройка параметров генерации с помощью предустановленных конфигураций. Вызывается при выборе значения в выпадающем списке пресетов.

Доступные пресеты:

- `custom` (Свой) – не меняет настройки, просто завершается.
- `min` (Минимальный) – `capitals=1`, `lowercase=1`, `digits=1`, `symbols=0`, длина=8.
- `std` (Стандартный) – `capitals=1`, `lowercase=1`, `digits=1`, `symbols=0`, длина=10.
- `max` (Максимальный) – `capitals=1`, `lowercase=1`, `digits=1`, `symbols=1`, длина=16.

После установки параметров вызывается `renderChk()` для обновления интерфейса и `saveCfg()` для сохранения настроек для текущего сервиса.

5.15. Функция `exportData()` – экспорт настроек в JSON

Назначение: выгрузка всех сохранённых настроек сервисов в JSON-файл для резервного копирования или переноса на другое устройство.

Порядок работы:

- 1) Функция создаёт новый объект `clean`, в который копируются настройки только для тех сервисов, которые есть в массиве `svcs`, а также для `'(empty)'`.
- 2) Формируется итоговый объект `data = { svcs: [...], perSvc: { ... } }`.
- 3) Объект преобразуется в JSON-строку с отступами (`JSON.stringify(data, null, 2)`) для удобочитаемости.
- 4) Создаётся `Blob` с типом `application/json`. Создаётся временная ссылка `<a>`, устанавливается `href = URL.createObjectURL(blob)`, `download = 'passgen_export.json'`. Ссылка программно кликается, после чего файл скачивается.

Формат экспортируемого файла:

```
{
"svcs": ["google", "github", "vk"],
"perSvc": {
"google": {
"state": {"capitals":1, "lowercase":1, "digits":0, "symbols":1},
"inv": {"cap":0, "dig":1},
"len": 16
}
}
}
```

- 12 -

5.16. Функция `importData()` – импорт из JSON

Назначение: загрузка настроек из JSON-файла, ранее созданного функцией `exportData()`.

Порядок работы:

- 1) Функция получает объект события (`e`) от скрытого поля ввода файла (`<input type='file' id='importFile'>`).
- 2) Извлекается первый выбранный файл (`e.target.files[0]`). Если файл не выбран, функция завершается.
- 3) Создаётся `FileReader`. При завершении чтения (`onload`) содержимое файла парсится как JSON. Если в результате нет полей `perSvc` или `svcs`, выводится сообщение 'Файл не содержит данных passgen'.
- 4) Данные сохраняются во временную переменную `importDataCache`, после чего вызывается `showImportModal(d)` для отображения модального окна выбора сервисов.
- 5) В случае ошибки парсинга JSON выводится сообщение 'Ошибка импорта: неверный формат файла'. Значение поля ввода файла сбрасывается.

5.17. Функция `showImportModal()` – окно выбора сервисов для импорта

Назначение: отображение модального окна со списком сервисов, найденных в импортируемом файле, для выбора конкретных сервисов для импорта.

Входной параметр: `d (object)` – объект с данными импорта (содержит `perSvc`).

Порядок работы:

- 1) Получение контейнера `modallist` и его очистка.
- 2) Извлечение списка сервисов из `d.perSvc`, сортировка по алфавиту.

- 3) Если сервисов нет – сообщение 'Нет сервисов для импорта' и завершение.
- 4) Для каждого сервиса создаётся label с чекбоксом (по умолчанию отмечен) и текстом – названием сервиса. Чекбокс получает атрибут data-svc с названием сервиса.
- 5) Модальному окну добавляется класс 'open' (становится видимым).

5.18. Функция `confirmImport()` – подтверждение импорта

Назначение: завершение импорта выбранных сервисов после нажатия кнопки 'Импортировать' в модальном окне.

Порядок работы:

- 1) Получение данных импорта из `importDataCache`. Если данных нет, окно закрывается.
- 2) Сбор всех отмеченных чекбоксов из `modallist`. Если ни один не отмечен, выводится сообщение 'Выберите хотя бы один сервис'.
- 3) Для каждого отмеченного сервиса: настройки копируются в `perSvc[svc]`; если сервиса нет в `svcs`, он добавляется.
- 4) Массив `svcs` сортируется, чипы перерисовываются. Если текущий сервис есть в импортированных, его настройки применяются.
- 5) Модальное окно закрывается. На 3 секунды отображается сообщение 'Импорт завершен!'

5.19. Функция `closeImportModal()` – закрытие окна импорта

Назначение: закрытие модального окна импорта и очистка временных данных при отмене импорта.

Порядок работы: у модального окна удаляется класс 'open' (становится невидимым), переменная `importDataCache` устанавливается в `null`.

- 13 -

5.20. Функция `toggleAdvanced()` – раскрытие расширенных настроек

Назначение: сворачивание/разворачивание панели расширенных настроек (Первая заглавная, Последняя цифра).

Порядок работы: функция переключает CSS-свойства панели между двумя состояниями. Если панель скрыта (`max-height = 0`, `opacity = 0`), она становится видимой (`max-height = 120px`, `opacity = 1`). Если видима – скрывается. Стрелка поворачивается на 90 градусов при раскрытии (трансформация `rotate`).

5.21. Функция `renderInv()` – отрисовка инверсий

Назначение: обновление визуального состояния кружочков расширенных опций (Первая заглавная, Последняя цифра) в соответствии с текущими значениями `window.inv`.

Порядок работы: для каждой опции ('cap', 'dig') находится соответствующий элемент по ID (`invCap`, `invDig`). Если `window.inv[опция]` истинно, элементу добавляется класс 'on' (кружок закрашивается фиолетовым), иначе класс 'on' удаляется (кружок пустой).

5.22. Функция `renderChk()` – отрисовка переключателей типов символов

Назначение: создание и обновление переключателей типов символов (Заглавные, Строчные, Цифры, Спецсимволы) в виде кружочков.

Порядок работы:

- 1) Функция очищает контейнер `chkGrid`.
- 2) Для каждого элемента из массива `CHK` (4 типа символов) создаётся контейнер с классом 'chk-item'. Внутри него – кружок (`span` с классом 'chk-circle') и текстовая подпись (название типа символов).
- 3) При клике на контейнер значение `window.state[id]` инвертируется (`0->1`, `1->0`) и вызывается `renderChk()` для перерисовки.
- 4) В зависимости от значения `state[id]` кружку добавляется или удаляется класс 'on'.
- 5) Функция вызывается: при инициализации, при смене сервиса, при применении пресета, при изменении расширенных настроек.

5.23. Функция `adj()` – регулировка длины пароля

Назначение: увеличение или уменьшение длины пароля на 1. Вызывается при нажатии на кнопки '+' (стрелка вверх) или '-' (стрелка вниз) рядом с полем длины.

Входной параметр: `d (number)` – шаг изменения (1 для увеличения, -1 для уменьшения).

Порядок работы: текущее значение длины считывается из поля 'length', к нему прибавляется `d`. Если результат меньше 1, устанавливается 1. Если больше 99, устанавливается 99. Границы защищают от ввода некорректных

значений.

5.24. Функция `toggleSvcPanel()` – панель популярных сервисов

Назначение: сворачивание/разворачивание панели с популярными и сохранёнными сервисами.

Порядок работы: функция переключает CSS-класс 'open' на контейнере `svcPanel` и на переключателе `svcToggle`. При открытии панель плавно появляется (`max-height` увеличивается, `opacity` становится 1), стрелка поворачивается. При закрытии – панель плавно исчезает.

- 14 -

5.25. Функция `renderSvcChips()` – отрисовка чипов сервисов

Назначение: создание и обновление всех чипов (кнопок) сервисов в панели популярных сервисов.

Порядок работы:

- 1) Функция очищает контейнер `svcGrid`.
- 2) Сначала отрисовываются все предустановленные популярные сервисы из массива `POPULAR`. Для каждого вызывается `chip(item.n, true, false)` – с логотипом и без возможности удаления.
- 3) Затем отрисовываются все пользовательские сервисы из массива `svcs`, которых нет среди `POPULAR`. Для каждого вызывается `chip(svc, false, true)` – без логотипа (отображается вопросительный знак) и с возможностью удаления по ПКМ.
- 4) Если пользовательских сервисов нет, отрисовывается только сообщение 'Нет сохранённых сервисов'.

5.26. Функция `chip()` – создание чипа сервиса

Назначение: создание одного чипа (кнопки) сервиса для панели сервисов.

Входные параметры:

- `item (string)` – название сервиса (например, 'google').
- `hasLogo (boolean)` – `true`, если сервис имеет предустановленный логотип; `false`, если нужно отобразить вопросительный знак.
- `deletable (boolean)` – `true`, если чип можно удалить по ПКМ; `false` для предустановленных сервисов (их удалять нельзя).

Порядок работы:

- 1) Создаётся `div`-элемент с классом 'svc-chip'. Ищется соответствующая запись в массиве `POPULAR` по названию (`item.n === name`).
- 2) Если сервис найден в `POPULAR`: создаётся кружок с первой буквой названия (или аббревиатурой), цвет фона кружка устанавливается из `POPULAR`. После кружка добавляется текстовый узел с названием сервиса.
- 3) Если сервис не найден в `POPULAR`: создаётся кружок с вопросительным знаком на сером фоне, добавляется текстовый узел с названием сервиса.
- 4) При клике на чип: название сервиса подставляется в поле ввода, панель сервисов закрывается, `curSvc` обновляется. Если для сервиса есть настройки – они применяются.
- 5) Если `deletable = true`: на чип вешается обработчик контекстного меню (`oncontextmenu`). При ПКМ сервис удаляется из массива `svcs`, чипы перерисовываются. Чип получает подсказку 'ПКМ для удаления'.
- 6) Функция возвращает созданный элемент для добавления в контейнер.

- 15 -

6. Обработка ошибок

В программе реализована многоуровневая обработка ошибочных ситуаций:

- Пустой сид. При попытке генерации без введённого сида поле подсвечивается розовой обводкой с анимацией пульсации, под полем появляется сообщение 'Введите сид'. Ошибка автоматически исчезает при начале ввода.
- Не выбран ни один тип символов. Если пользователь отключил все четыре переключателя, выводится сообщение 'Выберите хотя бы один тип символов'.
- Длина вне допустимого диапазона. Поле длины ограничено значениями от 1 до 99. При попытке выйти за эти границы значение принудительно корректируется функцией `adj()`.

- Неверный формат файла импорта. Если выбранный файл не является корректным JSON, выводится сообщение 'Ошибка импорта: неверный формат файла'.
- Отсутствие данных в файле импорта. Если JSON не содержит полей `perSvc` или `svcs`, выводится сообщение 'Файл не содержит данных `passgen`'.
- Нет сервисов для импорта. Если в файле импорта нет ни одного сервиса (`perSvc` пуст), выводится сообщение 'Нет сервисов для импорта'.
- Не выбран сервис при импорте. Если пользователь не отметил ни один сервис в модальном окне, выводится сообщение 'Выберите хотя бы один сервис'.
- Недоступность Clipboard API. Если современный метод копирования не сработал, используется запасной `execCommand('copy')`. Если и он не сработал, сообщение об ошибке не показывается, но пароль остаётся в поле для ручного копирования.

7. Экспорт и импорт (формат JSON)

Программа поддерживает экспорт и импорт настроек в формате JSON. Это позволяет переносить конфигурацию между устройствами или создавать резервные копии.

Структура экспортируемого файла:

```
{
  "svcs": ["google", "github", "vk"],
  "perSvc": {
    "google": {
      "state": {"capitals":1, "lowercase":1, "digits":0, "symbols":1},
      "inv": {"cap":0, "dig":1},
      "len": 16
    },
    "github": {
      "state": {"capitals":1, "lowercase":1, "digits":1, "symbols":1},
      "inv": {"cap":0, "dig":0},
      "len": 12
    }
  }
}
```

- `svcs` — массив строк с названиями сервисов, для которых были созданы пароли.
- `perSvc` — объект, где ключ — название сервиса, значение — объект с тремя полями:
- `state` — объект с флагами `capitals`, `lowercase`, `digits`, `symbols` (1 — включено, 0 — выключено).
- `inv` — объект с флагами `cap` (первая заглавная) и `dig` (последняя цифра).
- `len` — число, длина пароля.

При импорте пользователь может выбрать, какие сервисы импортировать. Если сервис уже существует, его настройки перезаписываются. Если сервис новый — он добавляется в список.

- 16 -

8. Безопасность

Программа разработана с учётом основных требований криптографической безопасности:

- Локальность вычислений. Весь код выполняется на стороне клиента (в браузере пользователя). Никакие данные (сид, пароли, настройки) не передаются по сети. Для работы программы не требуется интернет.
- Сессионный режим. При каждом запуске программы все ранее сохранённые данные очищаются. Это предотвращает утечку информации о том, для каких сервисов пользователь создаёт пароли, если злоумышленник получит доступ к компьютеру.
- Детерминированность. Пароль всегда можно восстановить заново, зная сид и название сервиса. Не нужно хранить пароли в файлах или облачных сервисах.
- Криптостойкость. Используется проверенный алгоритм HMAC-SHA256 с доказанной криптографической

стойкостью. Rejection Sampling гарантирует равномерное распределение символов в пароле.

- Отсутствие единой точки отказа. Компрометация одного пароля не раскрывает информацию о других паролях. Компрометация менеджера паролей невозможна, так как пароли не хранятся.
- Устойчивость к атаке length extension. Использование HMAC вместо простого SHA-256 предотвращает атаку добавления.
- Невозможность подбора. Без знания сида невозможно вычислить пароль даже при известном алгоритме генерации.

9. Запуск программы

Программа предназначена для работы на любых операционных системах (Windows, Linux, macOS), где есть современный браузер.

Способ запуска (единственный):

Откройте файл index.html в любом современном браузере (Google Chrome, Mozilla Firefox, Microsoft Edge, Safari, Opera, Yandex Browser). Для этого достаточно дважды кликнуть по файлу или перетащить его в окно браузера. После открытия файла программа полностью готова к работе. Все функции доступны сразу, никаких дополнительных действий не требуется.

Если вы используете Chromium-браузер (Google Chrome, Microsoft Edge, Opera), вы можете открыть программу в отдельном окне без адресной строки и вкладок. Для этого создайте ярлык браузера с параметром

--app=путь_к_index.html. Например:

```
google-chrome --app=/home/user/xxxulay/index.html --window-size=420,620
```

Программа не требует установки Python, Node.js или любых других дополнительных компонентов. Всё необходимое уже есть в вашем браузере.

- 17 -

10. Схема вызова функций при генерации

Ниже представлена последовательность вызова функций при нажатии кнопки 'Создать' (функция gen()) и кнопки 'Создать логин' (функция genLogin()).

10.1. Генерация пароля (нажатие кнопки 'Создать')

```
gen()
|
+-- Проверка seed (если пусто -> ошибка, выход)
+-- Проверка state (если все 0 -> ошибка, выход)
+-- Формирование алфавита из СНК
|
+-- hmacGen(seed, svc, len, alpha)
|
|
|
+-- importKey('raw', seed, 'HMAC-SHA256')
|
+-- Цикл while (pwd.length < len):
|
|
+-- msg = svc+len + counter++
|
|
+-- sign('HMAC', key, msg) -> 32 bytes
|
|
```

```

+-- for each byte:
|
|
if byte < max: pwd += alpha[byte % len(alpha)]
|
|
|
+-- return pwd
|
+-- Применение инверсий:
|
+-- Если inv.cap: первый символ -> toUpperCase()
|
+-- Если inv.dig: HMAC(seed+'_lastdig', svc+len) % 10 -> последний символ
|
+-- Вывод пароля в поле pwd
+-- updateStrength(len, sets)
+-- Копирование в буфер обмена
+-- Сохранение сервиса (если новый)
+-- saveCfg()

```

10.2. Генерация логина (нажатие кнопки 'Создать логин')

```

genLogin()
|
+-- Очистка поля логина
+-- Проверка seed (если пусто -> ошибка, выход)
+-- Проверка svc (если пусто -> выход)
+-- HMAC-SHA256(key=seed, msg='login/'+svc)
|
+-- Первые 6 байтов -> символы из alpha:
|
alpha = 'abcdefghijklmnopqrstuvwxyz23456789'
|
for i in 0..5: login += alpha[bytes[i] % len(alpha)]
|
+-- Вывод логина в поле login

```

- 18 -

11. Предустановленные популярные сервисы

В программу встроены следующие популярные сервисы с логотипами:

- 1) Google — буква 'g' на синем фоне (#4285f4). Крупнейшая поисковая система, также предоставляет почту Gmail, облачное хранилище Google Drive, видеохостинг YouTube и множество других сервисов.
- 2) GitHub — буквы 'GN' на тёмном фоне (#333). Крупнейший веб-сервис для хостинга IT-проектов и их совместной разработки, основан на системе контроля версий Git.
- 3) VK — буква 'V' на синем фоне (#4a76a8). Крупнейшая в Беларуси и России социальная сеть.
- 4) Yandex — буквы 'Ya' на красном фоне (#fc3f1d). Крупный белорусский и российский IT-холдинг, владелец поисковой системы, почты, такси, маркета и других сервисов.
- 5) Mail — символ '@' на синем фоне (#005ff9). Популярный почтовый сервис, также предоставляет облачное хранилище, мессенджер и другие сервисы.
- 6) iCloud — буква 'i' на тёмно-синем фоне (#369). Облачный сервис от Apple, предоставляющий синхронизацию данных между устройствами.
- 7) Onliner — буква 'O' на оранжевом фоне. Крупнейший белорусский портал, содержащий каталог товаров, форум,

новости и доску объявлений.

8) Kufar — буква 'К' на зелёном фоне. Крупнейшая белорусская онлайн-площадка для размещения частных объявлений о продаже товаров.

- 19 -

12. Константы и структуры данных

12.1. Массив СНК — наборы символов для пароля

```
СНК = [  
  ['capitals', 'Заглавные', 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'],  
  ['lowercase', 'Строчные', 'abcdefghijklmnopqrstuvwxyz'],  
  ['digits',  
   'Цифры',  
   '0123456789'],  
  ['symbols',  
   'Спецсимволы', '!@#$%^&*+=-~.'],  
];
```

Каждый элемент массива содержит: уникальный идентификатор (`id`), название на русском языке для отображения, строку со всеми символами данного типа. Обратите внимание: из заглавных и строчных букв исключены 'I', 'L', 'i', 'l', 'o', '0' — символы, которые легко перепутать с цифрами 1, 0. Из цифр исключены 0 и 1 — их путают с буквами.

12.2. Массив POPULAR — популярные сервисы

```
POPULAR = [  
  {n:'google', l:'g', c:'#4285f4'},  
  {n:'github', l:'GH', c:'#333'},  
  {n:'vk',  
   l:'V', c:'#4a76a8'},  
  {n:'yandex', l:'Ya', c:'#fc3f1d'},  
  {n:'mail',  
   l:'@', c:'#005ff9'},  
  {n:'icloud', l:'i', c:'#369'},  
  {n:'onliner', l:'O', c:'#ff6600'},  
  {n:'kufar',  
   l:'K', c:'#00a84d'}  
];
```

Каждый элемент содержит: `n` — название сервиса (строкой), `l` — буква или аббревиатура для отображения на логотипе, `c` — цвет фона логотипа в HEX-формате.

12.3. Глобальные переменные

`perSvc` — объект, хранящий настройки для каждого сервиса. Ключ — название сервиса, значение — объект с полями `state`, `inv`, `len`.

`curSvc` — строка, название текущего выбранного сервиса. По умолчанию `'(empty)'`.

`svcs` — массив строк, список всех сервисов, для которых когда-либо создавались пароли (кроме предустановленных).

`window.state` — объект с флагами `capitals`, `lowercase`, `digits`, `symbols` (1 — включено, 0 — выключено).

`window.inv` — объект с флагами `cap` (первая заглавная), `dig` (последняя цифра).

`importDataCache` — временное хранилище данных импорта между открытием модального окна и подтверждением.

Сбрасывается при отмене.

- 20 -

13. CSS-переменные и стили

Программа использует CSS-переменные (custom properties) для единообразного оформления. Основные переменные:

```
:root {
  --bg: #f5f3ff;
  /* Фон страницы (светло-фиолетовый) */
  --card: #fff;
  /* Фон карточек (белый) */
  --fg: #1a1a2e;
  /* Цвет текста (тёмный) */
  --border: #e5e7eb;
  /* Цвет границ (светло-серый) */
  --muted: #999;
  /* Приглушённый текст */
  --shadow: rgba(90,20,140,.08); /* Тень карточек */
  --chipbg: #ede9fe;
  /* Фон чипов */
  --hover: #ede9fe;
  /* Ховер */
  --pink: #f472b6;
  /* Розовый (слабый пароль) */
  --crimson: #e11d48;
  /* Малиновый (средний пароль) */
  --lilac: #a855f7;
  /* Сиреневый (сильный пароль, акцент) */
  --lilac2: #7c3aed;
  /* Тёмно-сиреневый */
  --violet: #8b5cf6;
  /* Фиолетовый */
  --error: #f43f5e;
  /* Цвет ошибки */
}
```

Анимации: fadeUp (плавное появление элементов снизу), glow (пульсация границы при ошибке), slideDown/slideUp (раскрытие панелей), checkPop (анимация переключения кружочка).

- 21 -

14. Полный список возможностей программы

Ниже перечислены все возможности программы passgen, реализованные в текущей версии:

- Генерация детерминированного пароля на основе HMAC-SHA256.
- Генерация логина на основе HMAC-SHA256.
- Регулировка длины пароля от 1 до 99.
- Выбор типов символов: заглавные, строчные, цифры, спецсимволы.
- Пресеты: Минимальный (8), Стандартный (10), Максимальный (16).
- Опция 'Первая заглавная' — преобразование первого символа в заглавный.
- Опция 'Последняя цифра' — замена последнего символа на цифру, вычисленную детерминированно.
- Индикатор надежности с тремя уровнями (розовый/малиновый/сиреневый).
- Автоматическое копирование пароля в буфер обмена.
- Сохранение индивидуальных настроек для каждого сервиса.
- Панель популярных сервисов с логотипами.
- Автоматическое добавление новых сервисов при генерации.
- Удаление сохранённых сервисов по правому клику мыши.
- Экспорт настроек в JSON-файл.
- Импорт настроек из JSON-файла с выбором сервисов.
- Сессионный режим (очистка данных при запуске).

- Анимированный интерфейс с плавными переходами.
- Кастомные переключатели в виде кружочков.
- Красивая цветовая схема с фиолетовыми градиентами.